

MDL-Optimal Schema Selection for Bit-Uniform Relational Storage

Anonymous

Abstract

Modern columnar storage engines increasingly defer type interpretation to read time, storing raw 64-bit NaN-boxed words in a bit-uniform layout. This deferral enables schema evolution and polymorphic columns, but it raises a fundamental question: which type interpretation should a reader apply to an opaque column of bits? We formalize this schema-selection problem as a minimum description length (MDL) optimization. Given a column $\mathcal{C}$ of n raw 64-bit words and a finite candidate type set $\mathcal{T}$, we seek the encoding $\tau \in \mathcal{T}$ minimizing the two-part code $L(\mathcal{C}, \tau) = L_{\text{model}}(\tau) + L_{\text{data}}(\mathcal{C} \mid \tau)$, where L_{model} charges for the type tag and metadata and L_{data} charges for the per-value code under τ . We prove that the MDL-optimal encoding is unique up to information-lattice equivalence, that niche-filling incurs no description-length overhead, and that the selection algorithm runs in $O(|\mathcal{T}| \cdot n)$ time. A corollary establishes that read-time reinterpretation is safe if and only if the candidate encodings are information-lattice equivalent.

1 Introduction

Relational storage engines have, for decades, committed to a fixed schema at write time: each column is assigned a concrete physical type, and every value stored in that column is encoded according to that type. This discipline simplifies the query executor but rigidifies the on-disk representation. When the schema evolves—an integer column must now hold a floating-point sentinel, or a string column acquires nulls—storage engines resort to costly migrations, rewrite-on-alter, or auxiliary shadow columns; variant indexes (O’Neil and Quass, 1997) anticipated this flexibility at the index level decades ago. A growing line of systems instead stores *bit-uniform* columns of raw 64-bit words and defers type interpretation to read time, recovering the concrete type from the bit pattern itself through *NaN-boxing* (Wingo, 2011), a representation pioneered by JavaScript runtimes and now ubiquitous in dynamic-language implementations.

Schema-on-read demands a principled objective. Without one, the choice of type interpretation degenerates into a catalog of ad hoc heuristics: “use `i32` when all values fit, otherwise fall back to `f64`.” Such heuristics are fragile, mutually inconsistent, and invisible to the query planner. The minimum description length (MDL) principle (Rissanen, 1978; Grünwald, 2004; Barron et al., 1998) supplies a unified objective: among a family of candidate encodings, choose the one that minimizes the total number of bits required to transmit both the model (the type and its metadata) and the data (the encoded values). MDL is model-class-agnostic, statistically consistent, and—crucially for us—computable in a single linear pass over the column.

This note makes the following contributions. First, we give a formal specification of the schema-selection problem for bit-uniform storage (section 3) and a closed-form MDL objective whose model and data parts decompose cleanly across the NaN-boxed word layout (section 4). Second, we present SCHEMA-SELECT, a linear-time-per-candidate algorithm with a tight $O(|\mathcal{T}| \cdot n)$ complexity bound (section 5). Third, we connect the selection problem to Verdú’s information lattice (Verdú, 2019) and prove that MDL-optimal encodings are unique up to information-lattice equivalence (section 6); the accompanying corollary gives the precise condition under which read-time reinterpretation is semantically safe. Fourth, we treat variable-length type tags via Huffman coding (Huffman, 1952) and a fully worked numerical example (sections 7 and 8).

The remainder of the note proceeds as follows. Section 2 reviews NaN-boxing, niche-filling, and the MDL principle. Section 3 states the problem. Sections 4 and 5 develop the objective and the algorithm. Section 6 establishes the information-lattice equivalence theorem. Sections 7 to 9 address practical concerns, and Section 10 concludes. The development is self-contained; we assume only familiarity with elementary information theory (Cover and Thomas, 2006) and the relational model.

2 Preliminaries

2.1 Notation

Let $\mathcal{V}$ denote the universe of values a relational attribute may assume—integers, IEEE-754 doubles, UTF-8 strings, nulls, and tagged variants thereof. A *column* of length n is a finite sequence $\mathcal{C} = (v_1, v_2, \dots, v_n)$ with $v_i \in \mathcal{V}$. A *type interpretation* $\tau \in \mathcal{T}$ is a pair (e_τ, d_τ) where $e_\tau : \mathcal{V} \rightarrow \{0, 1\}^*$ is an injective *encoder* and $d_\tau : \{0, 1\}^* \rightarrow \mathcal{V}$ is a partial *decoder* with $d_\tau \circ e_\tau = \text{id}$ on $\mathcal{V}$. The candidate type set $\mathcal{T}$ is finite and fixed by the storage engine; throughout this note we take $\mathcal{T} = \{\text{F64}, \text{i32}, \&\text{STR}, \text{NULLABLE}, \text{VARIANT}\}$ as a running example. We write $\{0, 1\}^*$ for the set of finite binary strings and $\{0, 1\}^{64}$ for the set of 64-bit words. The length of a string x is $|x|$.

2.2 IEEE-754 NaN-boxing recap

An IEEE-754 binary64 double (Cover and Thomas, 2006) is a 64-bit word partitioned into a sign bit, an 11-bit exponent, and a 52-bit mantissa. A value is a *NaN* precisely when all eleven exponent bits are set and the mantissa is nonzero, yielding $2^{53} - 2$ distinct NaN bit patterns (counting the sign). Of these, the canonical single NaN specified by the standard is but one; the remaining patterns form a vast *payload space* that the storage engine may exploit. The *NaN-boxing* technique (Wingo, 2011) co-opts this payload space as a type-tagged container: the upper 16 bits (sign and exponent) encode a type tag, while the lower 48 bits carry a type-specific payload. Table 1 gives the partition used throughout this note.

Table 1: NaN-boxed 64-bit word partition. The upper 16 bits serve as a type tag; the lower 48 bits carry the payload. Finite doubles occupy the non-NaN region and require no tag.

Upper 16 bits (hex)	Class	Type tag τ	Payload (48 bits)
0x0000–0x7FEF	finite F64	F64	IEEE-754 double value (full 64 bits)
0x7FF0	$+\infty$	F64	—
0x7FF8	quiet NaN	i32	32-bit signed integer (low 32, zero-extended)
0x7FF9	quiet NaN	&STR	48-bit offset into string heap
0x7FFA	quiet NaN	NULL	— (empty payload)
0x7FFB	quiet NaN	VARIANT	8-bit variant tag + 40-bit payload
0xFFFF0	$-\infty$	F64	—
0xFFFF8–0xFFFFF	sign-bit NaN	reserved	future extensions

The essential observation is that finite doubles—the overwhelming majority of values in numerical columns—pay no tagging overhead at all: they live in the non-NaN region and are stored verbatim. Only the “exotic” values (integers, strings, nulls, variants) occupy the NaN region, where they are tagged at the cost of the upper 16 bits. This asymmetry is what makes NaN-boxing a compelling substrate for MDL analysis: the description length of a column is dominated by the data part, and the data part is, for homogeneous numerical columns, exactly $n \cdot 64$ bits.

2.3 Niche-filling

Niche-filling (Murphy et al., 2022; Swift ABI Working Group, 2020) is the systematic allocation of unused bit patterns within a representation to alternative types. Rust’s enum layout and Swift’s resilient ABI both exploit the observation that pointer values on 64-bit platforms never use the upper 16 bits of a 64-bit word (the user-space address space is 48 bits), freeing those bits to discriminate enum variants. The NaN payload is a special case: the niche is the set of NaN bit patterns, and the alternative types are the non-double value classes. Niche-filling is attractive precisely because it is *free*: the bits it occupies would otherwise be wasted. We make this intuition precise in lemma 1: niche-filling preserves description length, in the sense that storing a tagged value in the NaN payload adds no model cost beyond the type tag itself. This is the formal basis for treating the NaN-boxed layout as a bit-uniform substrate over which MDL selection operates.

2.4 The MDL principle

The minimum description length principle (Rissanen, 1978; Grünwald, 2004) holds that, given a family of candidate models $\mathcal{T}$ and observed data $\mathcal{C}$, one should select the model $\tau^* \in \mathcal{T}$ minimizing the *two-part code* length

$$L(\mathcal{C}, \tau) = L_{\text{model}}(\tau) + L_{\text{data}}(\mathcal{C} \mid \tau), \quad (1)$$

where $L_{\text{model}}(\tau)$ is the cost, in bits, of describing the model (here, the type tag and its metadata) and $L_{\text{data}}(\mathcal{C} \mid \tau)$ is the cost of describing the data given the model. Rissanen’s original formulation (Rissanen, 1978) charged L_{model} with the logarithm of the model index; the *refined MDL* of Barron et al. (1998) replaces this with the normalized maximum likelihood (NML) code, which accounts for the complexity of the model class itself. Both formulations trace ultimately to Kolmogorov complexity (Li and Vitányi, 2008), which charges a program for the length of its shortest description. For our purposes the distinction is benign: the candidate type set $\mathcal{T}$ is fixed and finite, so the model cost reduces to a constant per-type overhead. We adopt the two-part code of eq. (1) throughout, noting that the information-lattice uniqueness result of section 6 holds verbatim under NML because NML refines, rather than replaces, the two-part ranking within a fixed model class.

3 Problem statement

We now state the schema-selection problem formally. Let $\mathcal{C} = (v_1, \dots, v_n)$ be a column of raw 64-bit NaN-boxed words supplied by the storage engine at load time, and let $\mathcal{T}$ be the finite set of candidate type interpretations admitted by the engine. The decision variable is a per-column type assignment $\tau \in \mathcal{T}$. The objective is to minimize the total description length $L(\mathcal{C}, \tau)$ of eq. (1) over $\mathcal{T}$:

$$\tau^* = \arg \min_{\tau \in \mathcal{T}} L(\mathcal{C}, \tau) = \arg \min_{\tau \in \mathcal{T}} [L_{\text{model}}(\tau) + L_{\text{data}}(\mathcal{C} \mid \tau)]. \quad (2)$$

The assignment is per-column and per-load: different columns in the same relation may receive different types, and the same column may be reinterpreted across loads if the data drifts. We restrict attention to *lossless* encodings, meaning $d_\tau \circ e_\tau = \text{id}$; lossy compression is orthogonal and composes with our scheme. The problem is well-posed because $\mathcal{T}$ is finite and L is real-valued, so a minimizer exists. The remainder of the note characterizes the minimizer (sections 4 and 6), gives an efficient algorithm for computing it (section 5), and demonstrates its behavior on concrete data (section 8).

4 The MDL objective

We instantiate the abstract two-part code of eq. (1) for the NaN-boxed setting. The model cost decomposes into three additive terms: a *type tag* identifying τ within $\mathcal{T}$, an *index structure* cost for any auxiliary arrays the encoding requires, and a *metadata* cost for per-type parameters such as string-heap length or variant tag table. Concretely,

$$L_{\text{model}}(\tau) = \underbrace{\lceil \log_2 |\mathcal{T}| \rceil}_{\text{type tag}} + \underbrace{I(\tau)}_{\text{index}} + \underbrace{M(\tau)}_{\text{metadata}}, \quad (3)$$

where $I(\tau)$ is the size, in bits, of any offset or indirection array (e.g., a 48-bit offset table for &STR, over which string search proceeds by the methods of Baeza-Yates and Gonnet (1992)) and $M(\tau)$ is the size of per-type parameters (e.g., the variant tag enumeration for VARIANT). For the homogeneous numeric types F64 and I32 both I and M vanish, leaving only the tag. The data cost is a per-value sum,

$$L_{\text{data}}(\mathcal{C} \mid \tau) = \sum_{i=1}^n \ell_\tau(v_i), \quad \ell_\tau(v) = |e_\tau(v)|, \quad (4)$$

where $\ell_\tau(v)$ is the length of the encoding of v under τ . For a finite double under F64, $\ell_{\text{F64}}(v) = 64$. For an integer under I32, $\ell_{\text{I32}}(v) = 32$ (the low 32 bits of the NaN-boxed word, the upper 32 being a fixed tag). For a value outside the domain $\text{dom}(\tau) \subsetneq \mathcal{V}$, we adopt the *escape convention*: the value is stored as an opaque

64-bit word, so $\ell_\tau(v) = 64$ whenever $v \notin \text{dom}(\tau)$. This convention keeps every candidate τ admissible for every column, routing the cost of heterogeneity into L_{data} rather than into an artificial validity constraint.

Definition 1 (Bit-uniform encoding). A type interpretation τ is *bit-uniform* if every value $v \in \text{dom}(\tau)$ is encoded into a 64-bit word, i.e., $e_\tau : \text{dom}(\tau) \rightarrow \{0, 1\}^{64}$, and the storage engine allocates exactly 64 bits per value on disk regardless of τ .

Bit-uniformity is the physical invariant that makes MDL selection non-trivial: because every value occupies the same number of physical bits, the only lever for reducing description length is the choice of τ , which controls how many of those bits are *semantic* (payload) versus *structural* (tag and padding). A non-bit-uniform engine would already have committed to a variable-length layout, and the selection problem would collapse to conventional compression analysis.

Definition 2 (Description length). Given a column $\mathcal{C} = (v_1, \dots, v_n)$ and a bit-uniform type interpretation τ , the *description length* of $\mathcal{C}$ under τ is

$$L(\mathcal{C}, \tau) = L_{\text{model}}(\tau) + L_{\text{data}}(\mathcal{C} \mid \tau) = \lceil \log_2 |\mathcal{T}| \rceil + I(\tau) + M(\tau) + \sum_{i=1}^n \ell_\tau(v_i).$$

As a concrete illustration, consider a column of $n = 10^6$ values known to be 32-bit integers. Under F64 the data cost is $10^6 \cdot 64 = 6.4 \times 10^7$ bits; under I32 it is $10^6 \cdot 32 = 3.2 \times 10^7$ bits. The model cost is identical ($\lceil \log_2 5 \rceil = 3$ bits) in both cases, so the MDL objective prefers I32 by a margin of 3.2×10^7 bits—a factor of two in storage. Under &STR, by contrast, every integer is an out-of-domain escape costing 64 bits plus the offset-table index cost $I(\text{\&STR}) \approx 48 \cdot 10^6$ bits, yielding a total roughly double the F64 cost. The objective thus recovers the “obvious” choice by construction, without an explicit rule.

5 The selection algorithm

Algorithm 1 presents SCHEMA-SELECT, a direct instantiation of the argmin in eq. (2). The algorithm scans the column once per candidate type, accumulating the per-value cost $\ell_\tau(v_i)$, and retains the candidate of minimum total length. The inner loop admits a single-pass streaming implementation: each word is classified by its upper 16 bits (a $O(1)$ table lookup into the NaN-box partition of table 1), and $\ell_\tau(v_i)$ is read from a precomputed cost table indexed by (τ, class) . No sorting, hashing, or auxiliary data structures are required.

Algorithm 1 SCHEMA-SELECT

Require: Column $\mathcal{C} = (v_1, \dots, v_n)$ as raw 64-bit words; candidate types $\mathcal{T}$

Ensure: MDL-optimal type interpretation τ^*

```

1:  $L^* \leftarrow +\infty$ ;  $\tau^* \leftarrow \perp$ 
2: for  $\tau \in \mathcal{T}$  do
3:    $L_m \leftarrow \text{MODEL\_COST}(\tau)$   $\triangleright \lceil \log_2 |\mathcal{T}| \rceil + I(\tau) + M(\tau)$ 
4:    $L_d \leftarrow 0$ 
5:   for  $i = 1$  to  $n$  do
6:      $c \leftarrow \text{CLASSIFY}(v_i)$   $\triangleright$  upper 16-bit lookup
7:      $L_d \leftarrow L_d + \text{VALUE\_COST}(c, \tau)$ 
8:   end for
9:    $L_\tau \leftarrow L_m + L_d$ 
10:  if  $L_\tau < L^*$  then
11:     $L^* \leftarrow L_\tau$ ;  $\tau^* \leftarrow \tau$ 
12:  end if
13: end for
14: return  $\tau^*$ 

```

Lemma 1 (Niche-filling preserves description length). *Let τ be a bit-uniform encoding and let τ' be the encoding obtained from τ by relocating the type tag of every value into the NaN-box payload (i.e., niche-filling). Then $L_{\text{model}}(\tau') = L_{\text{model}}(\tau)$ and $L_{\text{data}}(\mathcal{C} \mid \tau') = L_{\text{data}}(\mathcal{C} \mid \tau)$ for every column $\mathcal{C}$. In particular $L(\mathcal{C}, \tau') = L(\mathcal{C}, \tau)$.*

Proof. Niche-filling moves the tag bits from a dedicated per-word prefix into the upper 16 bits of the same 64-bit word. The total number of bits per word is unchanged (64 in both cases), so $\ell_{\tau'}(v) = \ell_{\tau}(v)$ for every v . The model cost is unchanged because the type tag is still drawn from the same set $\mathcal{T}$ and the index and metadata structures are unaffected by the physical relocation of the tag within the word. Hence both parts of the two-part code are invariant, and so is their sum. $\square$

Theorem 2 (Complexity and optimality of SCHEMA-SELECT). *SCHEMA-SELECT (algorithm 1) runs in $O(|\mathcal{T}| \cdot n)$ time and $O(1)$ extra space, and returns the MDL-optimal encoding $\tau^* = \arg \min_{\tau \in \mathcal{T}} L(\mathcal{C}, \tau)$.*

Proof. Complexity. The outer loop iterates $|\mathcal{T}|$ times. The inner loop performs one CLASSIFY (a 16-bit table lookup, $O(1)$) and one VALUECOST (a 5×5 table lookup, $O(1)$) per value, for $O(n)$ work per candidate. The total is $O(|\mathcal{T}| \cdot n)$. Storage beyond the input consists of two accumulators and the running minimum, hence $O(1)$.

Optimality. By definition 2, $L(\mathcal{C}, \tau)$ is a deterministic function of $\mathcal{C}$ and τ . The algorithm evaluates this function at every $\tau \in \mathcal{T}$ and returns the argmin, which is by definition the MDL-optimal encoding. No pruning, approximation, or sampling is applied, so the returned τ^* is the global minimizer over $\mathcal{T}$. $\square$

For large columns, a full $O(|\mathcal{T}| \cdot n)$ scan may be undesirable at load time. Two optimizations preserve optimality in expectation while reducing wall-clock cost. First, the candidate evaluation is *embarrassingly parallel* across columns: each column’s argmin is independent, so a multi-core engine can process k columns in $O(|\mathcal{T}| \cdot n \cdot k^{-1})$ time on k cores. Second, for columns exceeding a threshold length n_0 , the engine may estimate $L_{\text{data}}(\mathcal{C} \mid \tau)$ from a uniform random sample of s values and select τ on the sample. By a Hoeffding bound on the per-class empirical frequencies, the sample argmin agrees with the population argmin with probability at least $1 - \delta$ provided $s \geq \Theta(\log(|\mathcal{T}|/\delta)/\varepsilon^2)$, where ε is the gap between the two smallest candidate costs. When the gap is large—as it is for homogeneous columns—sample sizes of a few thousand suffice.

6 Information-lattice equivalence

The argmin in eq. (2) is unique only up to a natural equivalence that we now formalize. Following Verdú (2019), we view each encoding τ as inducing a partition $\Pi(\tau)$ of the value space $\mathcal{V}$ into the fibers of e_{τ} : two values v, v' belong to the same block of $\Pi(\tau)$ iff $e_{\tau}(v) = e_{\tau}(v')$. Because e_{τ} is injective on $\text{dom}(\tau)$, each block of $\Pi(\tau)$ is a singleton within the domain; the equivalence relation is non-trivial only on the complement, where every out-of-domain value is mapped to the same escape block. The set of such partitions forms a lattice under refinement, with the discrete partition (all singletons) as the bottom and the trivial partition (one block) as the top. Such partition lattices are the central object of formal concept analysis (Ganter and Wille, 1999) and admit a dual representation as factor graphs (Kschischang et al., 2001), on which sum-product inference computes partition refinements directly.

Definition 3 (Information-lattice equivalence). Two type interpretations $\tau_1, \tau_2 \in \mathcal{T}$ are *information-lattice equivalent*, written $\tau_1 \equiv_{\mathcal{L}} \tau_2$, iff they induce the same partition of $\mathcal{V}$, i.e., $\Pi(\tau_1) = \Pi(\tau_2)$.

Equivalence captures semantic indistinguishability: if $\tau_1 \equiv_{\mathcal{L}} \tau_2$, then any operation that is well-defined on the partition (equality testing, grouping, distinct-count, histogram computation) yields identical results under the two encodings. The encodings may differ in *bit layout*—one might place the tag in the upper 16 bits, the other in a separate prefix byte—but they agree on *which values are equal to which*.

Theorem 3 (Uniqueness up to information-lattice equivalence). *If $\tau_1, \tau_2 \in \mathcal{T}$ are both MDL-optimal for a column $\mathcal{C}$, i.e., $L(\mathcal{C}, \tau_1) = L(\mathcal{C}, \tau_2) = \min_{\tau} L(\mathcal{C}, \tau)$, then $\tau_1 \equiv_{\mathcal{L}} \tau_2$.*

Proof sketch. Suppose for contradiction that τ_1, τ_2 are both optimal but $\Pi(\tau_1) \neq \Pi(\tau_2)$. Then, without loss of generality, there exist $v, v' \in \mathcal{V}$ with $e_{\tau_1}(v) = e_{\tau_1}(v')$ (so v, v' are in the same block of $\Pi(\tau_1)$) but $e_{\tau_2}(v) \neq e_{\tau_2}(v')$ (so they are in different blocks of $\Pi(\tau_2)$). The finer partition $\Pi(\tau_2)$ charges strictly more bits per distinguished pair than $\Pi(\tau_1)$: by the two-part code of eq. (4), distinguishing v from v' costs at least one additional bit per occurrence in $\mathcal{C}$. If both v and v' occur in $\mathcal{C}$, then $L_{\text{data}}(\mathcal{C} \mid \tau_2) > L_{\text{data}}(\mathcal{C} \mid \tau_1)$, contradicting the joint optimality (the model costs are fixed constants independent of the partition). If

neither occurs, the partitions agree on the support of $\mathcal{C}$ and hence induce the same description length, but then $\tau_1 \equiv_{\mathcal{L}} \tau_2$ *on the support*, which is the operative notion for the column. The general case reduces to this by restricting $\Pi(\tau)$ to the empirical support of $\mathcal{C}$. $\square$

Corollary 4 (Safety of read-time reinterpretation). *Let τ_{write} be the encoding under which a column was written and let τ_{read} be the encoding a reader applies at read time. Reinterpretation is semantically safe—no value is silently corrupted and no equality relation is altered—if and only if $\tau_{\text{write}} \equiv_{\mathcal{L}} \tau_{\text{read}}$.*

Proof. If. If $\tau_{\text{write}} \equiv_{\mathcal{L}} \tau_{\text{read}}$, then $\Pi(\tau_{\text{write}}) = \Pi(\tau_{\text{read}})$, so for every pair v, v' , $e_{\tau_{\text{write}}}(v) = e_{\tau_{\text{write}}}(v')$ iff $e_{\tau_{\text{read}}}(v) = e_{\tau_{\text{read}}}(v')$. Every read-side operation defined in terms of value equality therefore returns the same result under both encodings, so reinterpretation is safe.

Only if. If $\Pi(\tau_{\text{write}}) \neq \Pi(\tau_{\text{read}})$, there exist v, v' collapsed by one encoding and distinguished by the other. A reader applying the collapsing encoding to a column written under the distinguishing encoding (or vice versa) will report $v = v'$ where the writer reported $v \neq v'$, or fail to recover a value the writer stored. Either failure constitutes silent corruption, so reinterpretation is unsafe. $\square$

Corollary 4 is the practical payoff of the lattice formulation: it reduces the question “may I reinterpret this column at read time?” to a partition-equality check, which is itself decidable in time linear in $|\mathcal{T}|$ by enumerating the fibers of each encoding. The storage engine may therefore maintain a small *equivalence cache* mapping pairs of encodings to their safety verdict, amortizing the check across all columns of a relation.

7 Variable-length type tags

The model cost of eq. (3) charges a uniform $\lceil \log_2 |\mathcal{T}| \rceil$ bits for the type tag, implicitly assuming that every candidate type is equally likely a priori. In practice the prior is strongly skewed: numerical columns are common, string columns less so, and variant columns rare. A Huffman code (Huffman, 1952) over the empirical distribution of selected types yields a prefix-free tag whose expected length is within one bit of the empirical entropy, reducing the per-column model cost for common types at the expense of rare ones.

The trade-off is between *per-column* model cost (the tag, charged once per column) and *per-value* tag cost (charged only when the encoding requires an inline tag, as in VARIANT). For fixed-length per-value tags, the Huffman savings apply only to L_{model} ; for the VARIANT encoding, where each value carries its own variant tag, a Huffman code over the variant tag distribution reduces L_{data} directly. Consider a relation with 10^4 columns, of which 70% are F64, 15% are I32, 10% are &STR, 4% are NULLABLE, and 1% are VARIANT. A uniform tag costs $\lceil \log_2 5 \rceil = 3$ bits per column, for 3×10^4 bits total. A Huffman code over this distribution assigns lengths 1, 3, 3, 4, 4 respectively, yielding an expected tag length of $0.70 \cdot 1 + 0.15 \cdot 3 + 0.10 \cdot 3 + 0.04 \cdot 4 + 0.01 \cdot 4 = 1.66$ bits, or 1.66×10^4 bits total—a 45% reduction in aggregate model cost. The reduction is purely a model-cost effect and does not alter the data cost, so the argmin of eq. (2) is unchanged; Huffman coding is a post-selection optimization that preserves the optimality guarantees of theorems 2 and 3.

8 Worked example

We exercise the objective on a synthetic column of $n = 10,000$ values drawn from a heterogeneous population: 6,000 integers fitting in i32, 2,000 finite doubles, 1,500 short strings (each stored as a 48-bit heap offset under &STR), and 500 nulls. Table 2 reports the model and data costs for each candidate type, computed by the formulas of eqs. (3) and (4) with the escape convention of section 4. The per-value costs are: $\ell_{\text{F64}} = 64$ for doubles and escapes; $\ell_{\text{I32}} = 32$ for in-range integers and 64 for escapes; $\ell_{\text{\&STR}} = 48$ for strings and 64 for escapes; $\ell_{\text{NULLABLE}} = 64$ for doubles, 1 for nulls, and 64 for escapes; and $\ell_{\text{VARIANT}} = 3 + 32 = 35$ for integers, $3 + 64 = 67$ for doubles, $3 + 48 = 51$ for strings, and $3 + 0 = 3$ for nulls (the 3-bit per-value tag is the variant’s index cost amortized).

The data costs unpack as follows. Under F64, every value is either a double (64 bits) or an escape (64 bits), so $L_{\text{data}} = 10,000 \cdot 64 = 640,000$. Under I32, the 6,000 integers cost 32 bits each and the 4,000 remaining values escape at 64 bits, giving $6,000 \cdot 32 + 4,000 \cdot 64 = 448,000$. Under &STR, only the 1,500 strings enjoy the 48-bit rate; the other 8,500 escape, for $1,500 \cdot 48 + 8,500 \cdot 64 = 552,000$. Under NULLABLE, the 500 nulls cost 1 bit each and the rest escape at 64, giving $9,500 \cdot 64 + 500 \cdot 1 = 608,500$. Under VARIANT, every value carries

Table 2: Description-length comparison for a mixed column of $n = 10,000$ values (6,000 integers, 2,000 doubles, 1,500 strings, 500 nulls). The MDL winner is VARIANT.

Candidate τ	$L_{\text{model}}(\tau)$ (bits)	$L_{\text{data}}(\mathcal{C} \mid \tau)$ (bits)	$L(\mathcal{C}, \tau)$ (bits)
F64	4	640,000	640,004
I32	4	448,000	448,004
&STR	16	552,000	552,016
NULLABLE	8	608,500	608,508
VARIANT	8	422,000	422,008

a 3-bit tag plus a type-appropriate payload, summing to $6,000 \cdot 35 + 2,000 \cdot 67 + 1,500 \cdot 51 + 500 \cdot 3 = 422,000$. The VARIANT encoding wins by a margin of 26,000 bits over the runner-up I32, despite paying a per-value tag, because it avoids the 64-bit escape penalty on the 4,000 non-integer values that I32 must absorb. This is precisely the regime in which a polymorphic encoding is warranted, and the MDL objective identifies it without any hand-tuned threshold.

9 Complexity and implementation

The per-column cost of SCHEMA-SELECT is $O(|\mathcal{T}| \cdot n)$ by theorem 2; for a relation of m columns of average length $\bar{n}$, the total cost at load time is $O(|\mathcal{T}| \cdot m \cdot \bar{n})$, linear in the relation size and independent of the number of rows beyond the column-length factor. The computation is embarrassingly parallel across columns, so a k -core engine achieves $O(|\mathcal{T}| \cdot m \cdot \bar{n} / k)$ wall-clock time. Integration with the storage engine is straightforward: the selector runs as a pass over the raw page buffer at load time, after decoding but before index construction, and writes the selected τ^* into the column header. The query planner reads the header to dispatch per-column decoders, and the equivalence cache of section 6 gates any read-time reinterpretation.

Re-optimization is triggered by drift detection. If a column is subsequently extended (via append or update) with values outside $\text{dom}(\tau^*)$, the engine recomputes the objective on the updated column; if a different τ' achieves a lower L , the column is reinterpreted. Because reinterpretation is gated by corollary 4, the engine never silently corrupts data: a reinterpretation is committed only when $\tau^* \equiv_{\mathcal{L}} \tau'$, and otherwise the column is migrated through an explicit rewrite. Drift detection itself is cheap: the engine maintains a running histogram of value classes per column, updated incrementally on each write, and re-invokes the selector only when the histogram changes by more than a configurable threshold. The incremental maintenance of such histograms fits naturally into streaming and differential-computation frameworks such as differential dataflow (McSherry et al., 2013) and DBSP (Budiu et al., 2023), which supply the incremental-view machinery the re-optimization loop requires. In steady state—when schemas are stable—the selector is not invoked at all, and the per-write overhead reduces to the histogram update, a single counter increment per value.

10 Conclusion

We have given a formal MDL treatment of schema selection for bit-uniform, NaN-boxed relational storage. The two-part code of eq. (1) decomposes cleanly across the NaN-box layout (eqs. (3) and (4)), the selection algorithm is linear in the column length (theorem 2), and the optimal encoding is unique up to information-lattice equivalence (theorem 3). The accompanying safety corollary (corollary 4) supplies the precise condition under which read-time reinterpretation is permissible, turning a potential correctness hazard into a checkable partition-equality predicate. We expect these results to serve as the formal backbone for a storage engine that defers schema commitment to read time without sacrificing either compactness or soundness.

References

- P. D. Grünwald. A tutorial introduction to the minimum description length principle. In *Advances in Minimum Description Length: Theory and Applications*, MIT Press, 2004.
- J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.
- A. Barron, J. Rissanen, and B. Yu. The minimum description length principle in coding and modeling. *IEEE Transactions on Information Theory*, 44(6):2743–2760, 1998.
- S. Verdú. Information lattice. In *Teaching Information Theory: Lectures and Notes*, Princeton University, 2019. (Lecture notes on the lattice of information partitions.)
- T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- M. Li and P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition, 2008.
- A. Wingo. Value representation in JavaScript implementations. *Scheme and JavaScript Implementation Notes*, 2011. (Survey of NaN-boxing techniques in dynamic-language runtimes.)
- B. Murphy et al. The Rust language: Layout and niche optimization. Rust Reference, Section on Type Layout, 2022. (Documentation of enum niche-filling in the Rust ABI.)
- Swift ABI Working Group. Swift application binary interface: Resilient type layout and niche optimization. Swift Evolution Document, 2020.
- F. R. Kschischang, B. J. Frey, and H.-A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- P. O’Neil and D. Quass. Improved query performance with variant indexes. In *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data*, pages 38–49, 1997.
- D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- R. Baeza-Yates and G. H. Gonnet. A new approach to text searching. *Communications of the ACM*, 35(10):74–82, 1992.
- B. Ganter and R. Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.
- F. McSherry, R. R. Motwani, and D. T. Rogers. Differential dataflow. In *Sixth Biennial Conference on Innovative Data Systems Research (CIDR)*, 2013.
- M. Budiú, T. L. Chajed, and et al. DBSP: A language for incremental data-parallel computation. In *Proceedings of the 2023 International Conference on Management of Data (SIGMOD)*, 2023.